

COMPETITION TEAMMATE FINDER

Revised Master Product, Architecture, Safety & Execution Plan

Objective: evolve the existing full-stack prototype into a focused, competition-aware platform that helps users form credible teams quickly, safely and measurably.

Product focus	Competition / hackathon team formation
Existing foundation	React + TypeScript • Node.js + Express • Prisma • PostgreSQL • Socket.io • JWT • AI
Core differentiator	Explainable teammate recommendations based on team gaps, skills, availability and reliability
New planning priorities	Cold-start strategy • event instrumentation • safety • competition governance • cost control • auth • privacy • realtime scaling

This revised plan incorporates the eight identified scope and risk gaps and converts them into concrete architecture, data models, milestones and acceptance criteria.

1. Executive Summary

Keep the idea. The existing implementation already has meaningful engineering substance: authentication, profiles, team management, AI-assisted functionality, similarity-based matching, persistent data and realtime chat. The main refinement is to make the product narrower and more rigorous.

Final product thesis: a participant chooses or joins a curated competition, states the role/skills their team needs, and receives explainable recommendations for complementary teammates. They can inspect credibility signals, invite candidates, form the team, communicate safely and eventually record the competition outcome.

The revised plan explicitly treats the following as first-class engineering concerns rather than future surprises: cold start, feedback instrumentation, user safety, multi-instance Socket.io, competition ownership, AI cost controls, complete authentication and privacy/data lifecycle.

2. Product Scope & Target User

Primary user: a student/developer who needs one or more teammates for a hackathon, coding competition, ideathon or similar team-based event.

Core job-to-be-done: “Help me find people who complement my team and are likely to actually participate.”

Core journey

- Discover: understand the product and choose a competition.
- Onboard: declare role, skills, experience, availability, interests and portfolio links.
- Define team gap: specify required roles/skills and preferred team size.
- Match: receive ranked candidates with reasons, not just a mysterious score.
- Trust: inspect profile evidence and activity/reliability signals.
- Invite: send a controlled invitation and receive accept/reject response.
- Collaborate: use a team workspace and realtime chat.
- Complete: record whether the team participated and optionally collect feedback.

Explicitly out of scope for the first public version

- General-purpose social feed, dating-style swiping, freelancer marketplace or generic networking.
- Autonomous AI team formation without user control.
- Advanced ML ranking trained on large datasets before there is sufficient outcome data.
- Large-scale competition scraping/import infrastructure.

3. Revised Production Architecture

```
competition-teammate-finder/
├── frontend/
│   ├── public/
│   └── src/
│       ├── app/                # router, providers, global config
│       ├── components/
│       │   ├── ui/
│       │   ├── forms/
│       │   ├── profile/
│       │   ├── matching/
│       │   ├── teams/
│       │   ├── chat/
│       │   ├── safety/
│       │   └── pages/
│       │       ├── Landing/
│       │       ├── Auth/
│       │       ├── Onboarding/
│       │       └── Competitions/
```

```

■      ■      ■■■ Dashboard/
■      ■      ■■■ Discover/
■      ■      ■■■ Profile/
■      ■      ■■■ Teams/
■      ■      ■■■ Notifications/
■      ■      ■■■ Settings/
■      ■■■ hooks/
■      ■■■ services/          # REST + Socket.io clients
■      ■■■ store/
■      ■■■ types/
■      ■■■ utils/
■
■■■ backend/
■    ■■■ src/
■      ■■■ config/
■      ■■■ middleware/
■      ■■■ routes/
■      ■■■ controllers/
■      ■■■ services/
■      ■      ■■■ auth.service.js
■      ■      ■■■ competition.service.js
■      ■      ■■■ profile.service.js
■      ■      ■■■ matching.service.js
■      ■      ■■■ team.service.js
■      ■      ■■■ invitation.service.js
■      ■      ■■■ moderation.service.js
■      ■      ■■■ analytics.service.js
■      ■■■ ai/
■      ■      ■■■ client.js
■      ■      ■■■ prompts.js
■      ■      ■■■ summarization.js
■      ■      ■■■ cache.js
■      ■■■ matching/
■      ■      ■■■ features.js
■      ■      ■■■ cold-start.js
■      ■      ■■■ scoring.js
■      ■      ■■■ explanations.js
■      ■      ■■■ evaluation.js
■      ■■■ analytics/
■      ■      ■■■ events.js
■      ■      ■■■ metrics.js
■      ■■■ socket/
■      ■      ■■■ auth.js
■      ■      ■■■ rooms.js
■      ■      ■■■ adapter.js
■      ■■■ server.js
■      ■■■ prisma/
■      ■      ■■■ schema.prisma
■      ■      ■■■ migrations/
■      ■      ■■■ seed.js
■      ■■■ tests/
■
■■■ docs/
■      ■■■ product.md
■      ■■■ architecture.md
■      ■■■ api.md
■      ■■■ matching.md
■      ■■■ safety.md
■      ■■■ privacy.md
■      ■■■ analytics.md
■■■ .env.example
■■■ docker-compose.yml
■■■ README.md
■■■ .github/workflows/ci.yml

```

Architecture rule: keep matching, safety, analytics and AI as domain services rather than embedding business logic inside HTTP route handlers. This makes the core behavior testable and replaceable.

4. Revised Data Model

The existing User/Profile/Team/TeamMember/Message foundation should be extended with competition context, invitations, moderation and event attribution.

Entity	Purpose	Important fields / notes
User	Identity + auth	id, email, passwordHash, role, status, emailVerifiedAt, createdAt

Entity	Purpose	Important fields / notes
Profile	Candidate information	bio, skills, roles, interests, experience, availability, portfolio URLs, aiSummary
Competition	Curated event	name, organizer, dates, teamSize, requiredSkills, status, createdBy, approval state
CompetitionMember	User ↔ event	userId, competitionId, status, joinedAt
Team	Competition team	competitionId, name, creatorId, status, targetSize
TeamMember	Membership + role	teamId, userId, role, joinedAt, status
TeamRequirement	What the team needs	teamId, role, skills, priority, filled status
TeamInvite	Controlled invitations	teamId, senderId, receiverId, status, createdAt, expiresAt
Message	Team chat	teamId, senderId, content, createdAt, moderation state
Recommendation	Attribution anchor	id, userId, candidateId, competitionId, score, scoreVersion, reasons, generatedAt
Recommendation Event	Feedback chain	recommendationId, eventType, actorId, timestamp, metadata
CompetitionOutcome	Outcome signal	teamId, participated, completed, result/placement optional, feedback
Report	Safety workflow	reporterId, targetUserId/messagingId, reason, status, reviewedBy, timestamps
Block	User protection	blockerId, blockedId, createdAt
ModerationAction	Admin enforcement	targetUserId, action, reason, adminId, createdAt, expiresAt
PasswordResetToken	Account recovery	userId, hashedToken, expiresAt, usedAt
Notification	Invites/system updates	userId, type, payload, readAt, createdAt

Privacy design requirement: define deletion behavior and cascade rules intentionally. A deleted user should not leave orphaned team membership, invitations or personal chat data without an explicit retention policy.

5. Cold-Start Strategy — P0

The system cannot wait for behavioral data before becoming useful. The matching engine therefore needs a deterministic fallback path from day one.

Three operating modes

Mode	When used	Signals
Cold Start	No/low interaction history	Declared skills, target role, competition fit, availability, experience, interests
Early Data	Some clicks/invites/acceptances	All cold-start signals + observed interaction features
Mature	Enough outcome data	Validated behavioral features + calibrated ranking, while retaining explainable base signals

Day-one seeding plan

- Curate the first competitions instead of allowing an empty public directory.
- Seed a small pool of credible starter profiles/test accounts for demos and early pilots.
- Allow an admin to manually feature or recommend initial candidates where the algorithm has no evidence.
- Use deterministic matching until enough recommendation events exist.
- Never fabricate behavioral/reliability history. Clearly distinguish declared profile information from observed activity.

Cold-start rule: absence of behavioral data must reduce reliance on behavioral signals, not prevent recommendations altogether.

6. Feedback Loop & Analytics Instrumentation — P0

Every recommendation must have a stable ID so the system can trace what happened after it was shown. This turns “we will tune the weights later” into an evaluable system.

Event	Required attribution	Purpose
recommendation_impression	recommendationId + userId	Candidate was shown
recommendation_open	recommendationId	Candidate profile inspected
recommendation_save	recommendationId	Candidate saved
invite_sent	recommendationId + teamId	Recommendation produced an invitation
invite_accepted/rejected	inviteId + recommendationId	Measures downstream conversion
team_joined	teamId + userId	Actual team formation
chat_started	teamId	Collaboration activation
competition_participated	teamId + competitionId	Real-world outcome
competition_completed	teamId + outcome	Success/feedback signal

Event	Required attribution	Purpose
report/block	target + context	Safety signal; separate from ranking feedback

Recommended chain

```

Recommendation(id=R123, score=82, scoreVersion=v1)
↓
recommendation_open(R123)
↓
invite_sent(R123, team=T9)
↓
invite_accepted(R123, invite=I77)
↓
team_joined(T9)
↓
competition_completed(T9, outcome=...)

```

Store a scoreVersion and the feature values/reasons used for the recommendation. Otherwise historical recommendations cannot be compared after the algorithm changes.

7. Safety Layer — P0/P1

This product connects strangers and provides realtime communication. Safety is therefore a core product requirement, not a cosmetic add-on.

MVP safety controls

- Report: report a user, team interaction or message with a structured reason.
- Block: immediately prevent direct interaction and suppress future recommendations where appropriate.
- Moderation queue: admin can inspect reports, evidence and history.
- Suspend/ban: temporary or permanent enforcement with audit records.
- Messaging limits: rate-limit invitations and messages, especially for new accounts.
- Abuse prevention: throttle repeated invitations, duplicate messages and rapid account creation.
- Authorization: users may only read/send messages in teams they belong to.
- Safety-aware ranking: blocked/suspended users must not be surfaced as recommendations.

Keep two distinct concepts

Trust / Candidate Quality	portfolio, activity, availability, experience, reliability indicators
Safety / User Protection	report, block, moderation, rate limits, suspension, message authorization

8. Competition Ownership & Curation — P0

Before building onboarding around a competition directory, define who is allowed to create competition records.

Recommended MVP governance

- Admin-curated competitions: only admins create/publish official competition entries in the first version.
- User submission later: users can suggest an event, but it enters a pending state and is not publicly matchable until approved.
- Organizer role later: verified organizers can manage their own events after an explicit verification workflow.
- Duplicate control: normalize name/date/organizer and warn about likely duplicates.

- Status model: draft → pending → approved → active → closed/archived

Why this is the recommended MVP choice: it prevents a brand-new marketplace from immediately filling with duplicate, fake or low-quality competition entries, while keeping the data model flexible enough for organizer functionality later.

9. AI Cost, Latency & Reliability Controls — P0/P1

AI summarization should improve the experience without becoming a required dependency for basic profile viewing or team formation.

Required controls

- Cache: persist the generated summary on the profile.
- Invalidate intelligently: regenerate only when relevant source fields change, such as bio/skills.
- Rate limit: cap user-triggered generations per time window.
- Timeout: set a bounded request timeout so pages do not hang.
- Fallback: if AI fails, display the original user-written bio/structured profile.
- Cost logging: record request count, latency, token/usage estimate where available and failure rate.
- Asynchronous generation: where possible, generate after onboarding/profile edits rather than blocking the profile save.
- Provider isolation: keep AI client code behind a service boundary so models/providers can be changed.

AI should not decide a user's worth. It can summarize and help interpret information; the core ranking should remain based on explicit, testable criteria.

10. Authentication & Account Security — P0

JWT + bcrypt is a starting point, not the complete authentication lifecycle.

Required before public deployment

- Email verification with a single-use, expiring token.
- Password reset via single-use, expiring token; store only a secure hash of the reset token.
- Rate-limit login, signup, password reset and verification endpoints.
- Use secure cookie-based token storage where appropriate for the deployment architecture; avoid exposing long-lived secrets to browser JavaScript.
- Validate password policy and all request bodies at the API boundary.
- Return generic authentication errors where detailed responses could enable account enumeration.
- Log security-relevant events without logging passwords, tokens or sensitive message content.

11. Privacy & Data Lifecycle — P0

Decide the lifecycle of profile, activity, recommendation and chat data before the schema becomes difficult to change.

Policy decisions to document

- Account deletion: define whether deletion is immediate, scheduled or subject to a limited retention period for abuse/audit records.
- Chat: define whether user-authored messages are deleted/anonymized when an account is deleted.
- Recommendations: retain aggregated analytics where useful, but avoid keeping unnecessary personal identifiers.
- Activity/reliability: distinguish user-declared information from system-observed behavior and explain what is shown publicly.
- Export: provide a reasonable path to export profile/account data if the project becomes public-facing.

- Visibility: allow users to control whether portfolio links, profile details and activity indicators are public to other participants.
- Retention: document retention periods for reports, moderation actions and operational logs

Design the Prisma relations and deletion behavior around these decisions. Do not rely on accidental database cascades.

12. Socket.io Scaling Plan — P1 / Deployment Gate

A single Express + Socket.io instance can use in-memory room state. Once multiple backend instances exist, broadcasts need shared coordination.

Current → scaled architecture

Single instance:

Client A  Express + Socket.io memory < Client B

Multiple instances:

Client A  API Instance 1 
 Redis Adapter  shared pub/sub
 Client B  API Instance 2 

- Do not scale Socket.io horizontally without a shared adapter.
- Plan for @socket.io/redis-adapter when multiple backend instances are introduced.
- Authenticate the socket connection and authorize room joins server-side.
- Persist important messages in PostgreSQL; Socket.io is the transport, not the source of truth.
- Make reconnect behavior idempotent so duplicate delivery does not create duplicate records.
- For the first MVP deployment, one backend instance is acceptable if documented as a deliberate limitation.

13. Matching Engine v2

The algorithm should evolve from simple similarity toward team-aware, explainable matching while preserving a deterministic fallback.

Recommended scoring dimensions

Signal	Initial importance	Implementation
Skill fit	High	Declared skills vs required skills
Role complementarity	High	Team gap vs candidate role
Competition fit	High	Same competition/category/context
Availability overlap	Medium-High	Time/day overlap
Experience fit	Medium	Declared experience level
Interests	Medium	Project/domain interests
Observed reliability	Medium, zero-weight at cold start	Response/activity signals once enough data exists

Illustrative baseline: 30% skill fit + 25% role complementarity + 15% competition fit + 10% availability + 10% experience + 10% reliability. Treat this as an initial hypothesis and version it as v1; evaluate changes rather than presenting the weights as objectively optimal.

Every recommendation should expose concise reasons such as “Strong backend fit”, “Matches your Saturday availability” or “Fills the team's ML requirement”.

14. Revised Execution Roadmap

Phase	Priority	Work	Exit criteria
0. Foundation	P0	Repository cleanup, env hygiene, schema review, roles/status enums, baseline tests	Clean reproducible local setup
1. Competition core	P0	Admin-curated competitions, approval/status flow, competition-aware onboarding	User can enter a real competition context
2. Matching core	P0	Team requirements, cold-start matcher, weighted scoring, explanations	Useful recommendations with no behavioral history
3. Feedback loop	P0	Recommendation IDs, event schema, score versions, funnel analytics	Recommendation → outcome chain is measurable
4. Safety	P0/P1	Report, block, moderation queue, suspension, message/invite limits	Abuse can be reported, contained and reviewed
5. Auth + privacy	P0	Email verification, reset, rate limits, deletion/retention rules, visibility controls	Public-demo security baseline complete
6. Trust layer	P1	Portfolio, availability, activity/reliability indicators	Candidate quality becomes easier to judge
7. AI hardening	P1	Caching, invalidation, rate limits, timeouts, fallback, usage logs	AI cannot block core UX or create uncontrolled cost
8. Realtime hardening	P1	Socket authorization, persistence, Redis adapter readiness	Safe single-instance deployment; documented scale path
9. Product polish	P1	Responsive UI, loading/error/empty states, accessibility, copy, demo data	Feels like a product, not a prototype
10. Ship + measure	P1	CI, deployment, monitoring, analytics review, demo video, README	Publicly demonstrable MVP
11. ML evolution	P2	Embeddings/ranking experiments only after outcome volume exists	Measured improvement over baseline

15. Definition of Done for the MVP

- A new user can register, verify email and complete a useful profile.
- An admin can create/approve a competition and define team size/requirements.
- A user can join a competition context and define the team gap.
- The matcher produces recommendations even with zero historical behavior.
- Every recommendation has an ID, score version and human-readable reasons.
- The system records the recommendation funnel through invitation and team formation.
- Users can report/block; admins can review reports and suspend accounts.

- Messaging is authorized, rate-limited and persisted.
- AI summaries are cached and fail gracefully.
- Account recovery and deletion behavior are implemented and documented.
- The application is deployed with secrets outside source control.
- Automated tests cover matching, authorization, invitations and safety-critical flows.

16. How to Evaluate Whether the Idea Is Working

Do not judge the project only by whether the UI looks good. Measure whether it solves the team-formation problem.

Core metrics

- Profile completion rate
- Recommendation open rate
- Invite rate per recommendation
- Invite acceptance rate
- Time from signup to first viable team
- Percentage of teams reaching target size
- Chat activation after team formation
- Competition participation/completion rate
- Report/block rate and moderation response time
- AI generation failure rate and average cost per active user

The matching experiment loop

1. Define scoring version v1
2. Collect attributed recommendation events
3. Compare conversion/outcome metrics
4. Inspect failure cases
5. Change one part of the scoring model
6. Ship as v2
7. Compare v2 against v1 on a defined evaluation period

Do not optimize solely for clicks. A useful recommendation should ultimately contribute to accepted invitations, viable teams and positive competition outcomes.

17. Technical Story for Resume / Interview

Competition Teammate Finder — Full-Stack Recommendation Platform
Built a competition team-formation platform using React, Node.js/Express, PostgreSQL/Prisma and Socket.io; implemented explainable, team-aware teammate recommendations using structured profile features and similarity scoring, with JWT authentication, realtime collaboration, moderation controls and AI-assisted profile summarization.

Strong interview topics

- Why a deterministic cold-start matcher was necessary.
- Why cosine similarity alone is insufficient for complementary team formation.
- How recommendation IDs create an end-to-end feedback loop.
- How scoreVersion enables evaluation across algorithm changes.
- How Socket.io authorization and Redis scaling work.
- How moderation and rate limiting protect a stranger-to-stranger platform.
- How AI caching and fallbacks control cost and latency.
- How privacy/deletion requirements influenced Prisma relationships.

18. Final Recommendation

Do not drop the project. The revised plan makes the idea materially stronger by solving the gaps that would otherwise appear when moving from a classroom prototype to a real product.

The most important changes are not “more features.” They are: competition context, cold-start matching, recommendation attribution, safety, governance, authentication completeness, privacy decisions, AI cost controls and a documented realtime scaling path.

If you execute only the P0 phases well, you already have a coherent and technically defensible project. The P1 work turns it into a polished portfolio product. P2 work should happen only when real usage provides a reason to make the matching model more sophisticated.

Decision	Plan
Keep the concept?	YES — refine, do not restart from scratch.
Primary positioning	Competition / hackathon team builder
Core technical story	Explainable recommendation + team-gap matching + realtime collaboration
Core product story	Find complementary, credible and available teammates quickly
Day-one strategy	Cold-start deterministic matching + curated competitions + starter profiles
Learning strategy	Instrument every recommendation and improve from measured outcomes
Safety strategy	Report + block + moderation + rate limits + suspension
Scaling strategy	Single instance initially; Redis adapter when horizontally scaling

Master principle: build the smallest system that can produce trustworthy team matches, measure what happens next, and improve from evidence.